MODULE 10 · DOMAIN 1 - MONITORING, LOGGING, ANALYSIS, REMEDIATION, AND PERFORMANCE OPTIMIZATION

Storage and Database Performance

S3, EFS, FSx, S3 Files, RDS, Aurora, DynamoDB and ElastiCache - matching the shape of the workload.

DOMAIN 1 of 5 EXAM WEIGHT 22% MODULE 10 of 35

admin@sthithapragna.com

Every storage and database service has a **natural shape**. Matching the workload to the shape beats tuning the wrong service for months.

S3 request performance

- **5,500 GET or HEAD** and **3,500 PUT, COPY, POST or DELETE** per second, **per partitioned prefix**.
- There is **no limit on the number of prefixes**. Ten prefixes give you ten times the throughput.
- Spread keys across prefixes when you need scale, rather than requesting a quota increase. There is no quota to raise.
- A **date-first key pattern** such as 2026/09/05/ concentrates every write of the day into one prefix.

> ON THE EXAM
>
> S3 scales horizontally through prefixes, not vertically. The design decision is the key naming scheme, and it is hard to change later.

Moving large objects quickly

- **Multipart upload**: recommended above 100 MB, **required above 5 GB**. Up to 10,000 parts, 5 TB maximum object.
- Parts upload in parallel, and a failed part is retried alone rather than restarting the whole object.
- **Byte-range fetches** do the same in reverse, downloading ranges of one object in parallel.
- Incomplete multipart uploads **keep being billed**. A lifecycle rule to abort them after 7 days is standard practice.

> ON THE EXAM
>
> The abort-incomplete-multipart lifecycle rule is a genuine cost answer. Failed uploads leave parts that never appear in the object listing.

Transfer Acceleration or DataSync?

TRANSFER ACCELERATION

Long-distance uploads to S3

- Routes over the CloudFront edge network
- For clients geographically far from the bucket
- A property of the bucket, no agent needed
- Charged per GB transferred

DATASYNC

Bulk transfer between file systems

- On-premises NFS or SMB to S3, EFS or FSx
- Handles scheduling, verification and retries
- Agent-based for on-premises sources
- For migrations and recurring sync jobs

THE TIEBREAKER

One client uploading from far away: Transfer Acceleration. A file server's worth of data moving into AWS: DataSync.

S3 Express One Zone

- A storage class using **directory buckets**, in a **single Availability Zone** you choose, co-located with your compute.
- **Single-digit millisecond** first-byte latency, up to **10 times faster** than S3 Standard.
- Default **200,000 reads and 100,000 writes per second** per directory bucket, raisable to millions.
- **One AZ only.** Losing that AZ loses the data, so it is for hot working sets rather than the record copy.

> ON THE EXAM
>
> Higher storage price, much lower request price. It wins on workloads with huge numbers of small requests, not on quiet archives.

Shared file storage: which one

SERVICE	CHOOSE IT WHEN
Amazon EFS	Linux, NFS, many clients, elastic capacity. The default for Linux shared storage
Amazon S3 Files	You want an existing S3 bucket mounted as NFS, with S3 API access preserved
FSx for Windows File Server	SMB, Windows, Active Directory integration, NTFS ACLs
FSx for Lustre	HPC and machine learning at very high throughput, linked to an S3 bucket
FSx for NetApp ONTAP	Multi-protocol NFS and SMB, snapshots, dedup, existing ONTAP skills
FSx for OpenZFS	NFS with very low latency and ZFS snapshot semantics

ON THE EXAM

The protocol usually decides it. SMB and Active Directory means FSx for Windows. NFS on Linux means EFS unless something else is named.

Amazon S3 Files, added in exam guide v1.1

- Generally available **April 2026**. Mounts an S3 bucket as a shared **NFS v4.2** file system, and it is built on Amazon EFS.
- **Both access paths work at once.** The same data is reachable through the file system and through the S3 API.
- It **caches the active working set**; large reads of 1 MB and above stream from S3 at ordinary S3 GET cost.
- Named in **skill 1.3.4** alongside EFS and FSx, so it is explicitly examinable.

> ON THE EXAM
>
> "A legacy application needs a POSIX file path, but the data must stay in S3 for the analytics pipeline" is exactly this service.

EFS throughput and performance modes

- **Elastic throughput is the default**. It scales automatically and you pay for what you read and write. Best for spiky workloads.
- **Bursting** ties throughput to how much you store, using a credit balance. **Provisioned** fixes a rate you pay for regardless of use.
- **General Purpose** performance mode gives the lowest latency and is right roughly 90 percent of the time.
- **Max I/O is previous generation**, higher latency, and is **not supported with One Zone or with Elastic throughput**.

> ON THE EXAM
>
> Performance mode cannot be changed after creation. Throughput mode can. Getting the first one wrong means a new file system and a migration.

EFS storage classes and cost

- **Standard**, **Infrequent Access** and **Archive**. Lifecycle management moves files between them automatically.
- A typical policy: move to Infrequent Access after **30 days** without access, and to Archive after **90**.
- Reading from Infrequent Access or Archive incurs a **per-GB access charge**. Frequently read files should not be tiered down.
- **Regional** file systems span Availability Zones. **One Zone** is cheaper and loses data if that zone is lost.

> ON THE EXAM
>
> EFS lifecycle is the biggest single cost lever on a large file system, and the access charge is the reason not to set it too aggressively.

RDS monitoring: three separate layers

- **CloudWatch metrics** come from the hypervisor: CPU, connections, storage, IOPS. The outside view.
- **Enhanced Monitoring** runs an agent on the database host for **OS-level** metrics, as often as **1 second**. It writes to **CloudWatch Logs**.
- **CloudWatch Database Insights** shows **database load** by wait event and by SQL statement. The inside view.
- Enhanced Monitoring metrics are **not** ordinary CloudWatch metrics, so alarming on them means a metric filter first.

> ON THE EXAM
>
> "CPU is fine but the database is slow" needs the inside view. Only database load by wait event tells you what queries are waiting on.

Currency: Performance Insights has been replaced

- **RDS Performance Insights reached end of life in 2026** and the console now redirects to **CloudWatch Database Insights**.
- **Standard mode** preserves the old experience and pricing, with 7 days of detail included.
- **Advanced mode** adds **fleet-level monitoring, lock diagnostics and execution plan capture**, which Performance Insights never had.
- The **Performance Insights API still exists unchanged**, so CloudFormation and Terraform keep working.

> ON THE EXAM
>
> Every SOA course written before 2026 teaches "RDS Performance Insights". Learn the capability under both names, and expect Database Insights in the console.

RDS scaling levers

- **Storage autoscaling** grows the volume when free space runs low, up to a maximum you set. It never shrinks.
- **Modify the instance class** for CPU and memory. On Multi-AZ, the standby is done first, so downtime is one failover.
- **Parameter groups** tune the engine. **Static** parameters need a reboot; **dynamic** ones apply immediately.
- RDS storage is EBS underneath, so **gp3** and provisioned IOPS behave exactly as module nine described.

> ON THE EXAM
>
> Storage autoscaling prevents the storage-full outage. It does nothing for IOPS, which is a separate setting on the volume.

RDS Proxy

- A managed **connection pool** in front of RDS or Aurora. Applications connect to the proxy, not the database.
- Solves **connection exhaustion**, which is the classic Lambda-plus-RDS failure: thousands of functions, thousands of connections.
- Reduces **failover time** substantially, because the proxy holds client connections open while the database fails over.
- Integrates with **Secrets Manager** and supports **IAM authentication**, so credentials leave the application.

> ON THE EXAM
>
> "Too many connections" errors from a serverless front end is RDS Proxy. A bigger instance buys a few hundred more connections and delays the same failure.

Aurora, and Serverless v2

- Storage is a **shared cluster volume**, six copies across three AZs, growing automatically. Replicas read the same volume.
- Up to **15 read replicas** with typically sub-second replica lag, because there is no log shipping.
- **Serverless v2** scales in fine increments measured in **Aurora Capacity Units**, each roughly 2 GiB of memory.
- It can scale **down to zero ACUs** and pause, then resume on the next connection. That is the development-environment answer.

> ON THE EXAM
>
> Aurora replicas share one storage volume, so adding a replica adds no storage cost and almost no replication lag. RDS replicas each get their own copy.

DynamoDB: on-demand or provisioned?

ON-DEMAND WHEN

You cannot predict load

- New applications with no traffic history
- Spiky, seasonal or unpredictable patterns
- No capacity planning and no throttling from under-provisioning
- Higher price per request

PROVISIONED WHEN

Load is predictable

- Steady, well understood traffic
- Substantially cheaper per request at scale
- Pair with auto scaling for daily cycles
- Reserved capacity available for further savings

THE TIEBREAKER

You can switch between modes, but only once every 24 hours. That limit appears in questions about reacting to an unexpected event.

When DynamoDB throttles

- **A hot partition.** A partition supports about **3,000 read units and 1,000 write units** per second, whatever the table is provisioned for.
- **Adaptive capacity** shifts throughput toward busy partitions automatically, but it cannot fix a badly chosen partition key.
- **A throttled GSI throttles the base table's writes**, because writes must succeed on both.
- The fix is nearly always the **partition key design**: higher cardinality, or a write-sharding suffix.

> ON THE EXAM
>
> Throttling on a table that is well under its provisioned capacity is a hot partition. More capacity will not help; a better key will.

DAX or ElastiCache?

DAX WHEN

The database is DynamoDB

- Purpose-built DynamoDB cache, microsecond reads
- Write-through, so it is API-compatible
- Caches items and query results
- No application caching logic to write

ELASTICACHE WHEN

Anything else, or more control

- Caches RDS, Aurora, computed results, sessions
- Valkey and Redis, or Memcached
- You control eviction, TTL and structure
- Application code manages the cache

THE TIEBREAKER

DAX only accelerates DynamoDB, and needs almost no code change. ElastiCache works with anything, and needs code.

ElastiCache: which engine

- **Valkey and Redis**: rich data structures, persistence, replication, cluster mode, pub/sub, sorted sets. Valkey is the open-source continuation.
- **Memcached**: simple key-value, multi-threaded, no persistence and no replication.
- **Cluster mode enabled** shards data across node groups, which is how you scale writes as well as reads.
- Needing **persistence, replicas, failover or anything beyond strings** rules Memcached out immediately.

> ON THE EXAM
>
> Memcached is the answer only when the requirement is a simple, disposable cache that scales by adding identical nodes.

A legacy reporting application must read data files from a POSIX file path. The same files are already in an S3 bucket and are read continuously by an analytics pipeline that uses the S3 API. Copying the data would create two versions that drift.

The application runs on Linux EC2 instances and cannot be modified.

What should the team use?

A Amazon S3 Files, mounting the existing bucket as an NFS file system

B DataSync on a schedule, copying the bucket into an EFS file system

C FSx for Lustre linked to the bucket, with the application mounting Lustre

D Mountpoint for Amazon S3 on each instance

◆ PAUSE AND DECIDE

What should the team use?

A Amazon S3 Files

Purpose-built for this: one copy of the data, reachable through NFS and the S3 API at the same time. No drift and no application change.

B DataSync into EFS

Creates the second copy the requirement rules out. Scheduled sync means the two versions are different between runs.

C FSx for Lustre

Genuinely links to S3 and is built for HPC throughput, at HPC cost. It is heavy machinery for a reporting application.

D Mountpoint for S3

The strongest alternative, and it is not fully POSIX. Random writes and some file operations are unsupported, which is a risk for unmodifiable legacy code.

THE SIGNAL

"Copying would create two versions that drift" is the clause. Only simultaneous dual access to one copy satisfies it.

> A DynamoDB table is provisioned at 20,000 write capacity units and averages 6,000. Users report intermittent failures, and CloudWatch shows WriteThrottleEvents throughout the day. The partition key is a country code, and roughly 80 percent of traffic is from one country.
>
> The team has already tried increasing provisioned capacity to 30,000, which did not help.

What is the correct fix?

A. Switch the table to on-demand capacity mode

B. Redesign the partition key for higher cardinality, for example by adding a shard suffix

C. Add a DAX cluster in front of the table

D. Enable DynamoDB auto scaling with a higher maximum

◆ PAUSE AND DECIDE

What is the correct fix?

A. Switch to on-demand

On-demand removes provisioning, not per-partition limits. One partition still caps at about 1,000 write units per second.

B. Redesign the partition key

The traffic is concentrated on one partition key value. Only spreading writes across more partitions raises the effective ceiling.

C. Add DAX

DAX caches reads. The throttling here is on writes, which pass straight through to the table.

D. Auto scaling with a higher maximum

The same mistake the team already made, automated. More table capacity does not raise the per-partition limit.

THE SIGNAL

The scenario states the experiment and its result: more capacity did not help. That is the evidence that the limit is not table-level.

A serverless API on Lambda connects to an Aurora PostgreSQL cluster. Under load the functions return "too many connections" errors, although the database's CPU and memory are barely used. Scaling the writer to a larger instance class helped for a fortnight, then the errors returned.

A failover last month also caused eight minutes of errors while connections re-established.

What should the team implement?

A RDS Proxy in front of the Aurora cluster

B A larger instance class again, with storage autoscaling enabled

C Aurora Serverless v2 with a higher maximum ACU setting

D ElastiCache in front of the database to reduce query volume

◆ PAUSE AND DECIDE

What should the team implement?

A RDS Proxy

Pools connections so thousands of Lambda executions share a few, and holds client connections open through a failover. It answers both symptoms.

B Bigger instance again

The scenario already ran this experiment. A larger class raises the connection limit somewhat and delays the same failure.

C Serverless v2 with higher ACUs

Scales compute, and the database is not short of compute. CPU and memory are stated as barely used.

D ElastiCache

Reduces query volume, which helps a busy database. It does not change how many connections Lambda opens.

THE SIGNAL

Two symptoms, one cause. Connection exhaustion with idle CPU, plus slow failover recovery, are both what a connection proxy exists to fix.

> An ingestion pipeline writes about 4,000 small objects per second into an S3 bucket. Keys are formatted as logs/2026/09/05/HH-MM-SS-uuid.json. The team sees 503 Slow Down responses and rising retry rates, and has asked AWS Support to raise the bucket's request quota.
>
> Reads are performed later by a batch analytics job.

What should the team do?

A. Wait for the quota increase, since 4,000 writes per second exceeds the bucket limit

B. Change the key scheme so writes spread across many prefixes, for example by leading with a hash or a shard identifier

C. Enable S3 Transfer Acceleration on the bucket

D. Switch the bucket to the S3 Express One Zone storage class

◆ PAUSE AND DECIDE

What should the team do?

A. Wait for a quota increase

There is no per-bucket request quota to raise. The 3,500 writes per second figure is per prefix, and prefixes are unlimited.

B. Spread writes across prefixes

A date-first key puts every write of the day in one prefix. Leading with a hash or shard spreads them, and throughput scales with prefix count.

C. Transfer Acceleration

Improves long-distance transfer speed. It does nothing about request rate, and the pipeline is presumably already in-Region.

D. S3 Express One Zone

Would raise the ceiling substantially, at higher storage cost and single-AZ durability. It also does not fix a key design that will concentrate load anywhere.

THE SIGNAL

The team's support request is the distractor. Knowing which limits are per-prefix rather than per-bucket is what turns this from a quota problem into a design one.

The five things to carry forward

- **S3 scales by prefix, not by quota.** A date-first key scheme is a self-inflicted throughput cap.
- **Protocol picks the file service.** SMB means FSx for Windows; S3 Files mounts a bucket as NFS without copying it.
- **Performance Insights is now CloudWatch Database Insights.** Learn the capability under both names.
- **Connection errors with idle CPU means RDS Proxy.** A bigger instance delays the same failure.
- **Throttling below provisioned capacity is a hot partition.** Fix the key, not the capacity.

> ON THE EXAM
>
> Next: observability beyond CloudWatch - X-Ray, Managed Prometheus, Managed Grafana and the Health Dashboard.